Ethereum

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Arnaldo Tomasino

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Foundational Concepts of Ethereum **5**
1.1 Defining Ethereum: The World Computer . . . 5
1.2 Core Applications: Smart Contracts and Decentralized Applications (Dapps) . . 5
1.3 The Ethereum Virtual Machine (EVM) . . . 5

2 The Genesis and Motivation of Ethereum **7**
2.1 A Brief History . . . 7
2.2 Why Ethereum? Analyzing the Limitations of Bitcoin . . . 7

3 The Ethereum Account Model **9**
3.1 The Two Types of Accounts . . . 9
3.2 The Structure of an Account . . . 9

4 The Mechanics of Execution: Transactions, Messages, and Gas **11**
4.1 Transactions: The Catalyst for State Change . . . 11
4.1.1 Clarifying the transaction signature . . . 12
4.2 Messages: Contract-to-Contract Communication . . . 13
4.3 Gas: The Fuel for the Ethereum Network . . . 13
4.3.1 Clarification of figure 4.2 . . . 14

5 The Ethereum State Transition Function **15**
5.1 The State Transition Process . . . 15

6 Data Structures: Blocks and Merkle-Patricia Trees **17**
6.1 The Structure of an Ethereum Block . . . 17
6.2 The Merkle-Patricia Tree . . . 17

7 Defining Token Systems on Ethereum **21**
7.1 Token Standards: A Framework for Interoperability . . . 21
7.1.1 ERC-20: The Standard for Fungible Tokens . . . 21
7.1.2 ERC-721: The Standard for Non-Fungible Tokens (NFTs) . . . 22
7.2 The Evolution of the Ethereum Protocol . . . 22
7.2.1 The Merge: Transition to Proof of Stake . . . 22
7.2.2 Sharding: A Proposed Solution for Scalability . . . 22

1 Foundational Concepts of Ethereum

Ethereum is a general-purpose, programmable blockchain that extends beyond the capabilities of earlier platforms designed primarily for financial transactions. Understanding its fundamental architecture—the platform itself, its native currency, and its execution environment—is essential before exploring its more complex mechanics and applications. This chapter provides a foundational overview of the Ethereum protocol, from its core concepts and motivations to the specific data structures that ensure its integrity.

1.1 Defining Ethereum: The World Computer

Ethereum is the world's largest programmable blockchain platform. It is an open-source project distributed under the GPLv3 license. Conceived as a "world computer," Ethereum is a general-purpose platform designed to support a wide range of applications that go far beyond simple financial transactions. Its native cryptocurrency, Ether (ETH), is not only a medium of exchange but also serves as the fuel for powering computation across the network.

1.2 Core Applications: Smart Contracts and Decentralized Applications (Dapps)

The core logic executed on the Ethereum blockchain is encapsulated in **smart contracts**. These immutable pieces of code are stored on the blockchain and run as programmed. Applications that use the blockchain as a back-end, leveraging smart contracts for their core functionality, are known as **Decentralized Applications (Dapps)**.

Because their logic is executed on-chain, Dapps possess three key properties:

- **Trustworthiness:** Once deployed on Ethereum, a Dapp's smart contracts will run exactly as programmed, ensuring deterministic outcomes for given inputs.
- **Asset Control:** Dapps can be designed to manage and control digital tokenized assets with a high degree of precision.
- **Decentralization:** By their nature, Dapps are not controlled by any single entity or person.

The main application categories built on Ethereum include cryptocurrency wallets, financial applications, decentralized markets, and games where players can own and trade in-game assets.

1.3 The Ethereum Virtual Machine (EVM)

At the heart of Ethereum is the **Ethereum Virtual Machine (EVM)**, the execution environment where all smart contract code runs. Every node participating in the Ethereum network

runs an instance of the EVM.

This design ensures that all nodes execute the exact same code, which is the mechanism for validating transactions and confirming state changes across the entire distributed system. Having established what Ethereum is at a high level, it is useful to understand why it was created and how it evolved from its predecessors.

2 The Genesis and Motivation of Ethereum

Ethereum was not developed in a vacuum; it was created as a direct response to the perceived limitations of earlier blockchain platforms, most notably Bitcoin. Its design was motivated by a desire to build a more flexible and general-purpose blockchain.

2.1 A Brief History

Ethereum's development can be traced through several key milestones:

- **2013:** Vitalik Buterin first proposed the concept in a white paper.
- **December 2013:** Buterin, along with four other founders, established the Ethereum.org non-profit organization.
- **2014:** The founders established Ethereum Switzerland GmbH and began software development.
- **July 30, 2015:** The first stable release of the Ethereum software was launched.
- **Post-2015:** The protocol has undergone several changes through soft and hard forks to add new features, combat attacks, and improve overall security.

2.2 Why Ethereum? Analyzing the Limitations of Bitcoin

The development of Ethereum was driven by the goal of overcoming four primary limitations observed in the Bitcoin protocol.

1. **Lack of Turing-Completeness** A computational system is considered Turing-complete if it can perform a wide range of general-purpose operations, such as loops and conditionals, allowing it to implement complex algorithms. Bitcoin's scripting language is not Turing-complete; for example, it completely lacks constructs for loops. This severely limits its functionality. Ethereum was designed with more general-purpose scripting languages, like Solidity, which are Turing-complete and enable the development of far more sophisticated applications.
2. **Value-Blindness** Bitcoin's Unspent Transaction Output (UTXO) model is "all-or-nothing." A UTXO script cannot provide fine-grained control over the amount of currency that can be withdrawn from an output; the entire UTXO must be consumed in a transaction. In contrast, Ethereum's account-based model allows for more direct and precise value transfers, where any desired amount can be sent from an account's balance, provided the funds are sufficient.
3. **Lack of State** The all-or-nothing nature of the UTXO model means that multi-stage contracts or scripts cannot maintain an internal state between transactions. For example, a contract that requires a timeout or depends on multiple conditions being met over time is not feasible. Ethereum solves this with stateful smart contracts, where data can be stored and updated within the contract itself, enabling advanced protocols and applications.

4. **Blockchain-Blindness** In Bitcoin, UTXO-based transactions cannot access blockchain data such as the nonce or the hash of a previous block. Ethereum allows smart contracts to access this on-chain data, which can be used as a source of randomness for applications like games and gambling.

These motivations led to a fundamental architectural shift away from the UTXO model and toward an account-based system.

3 The Ethereum Account Model

Unlike Bitcoin's UTXO ledger, Ethereum's state is defined by the properties of all existing accounts. Every change to the network is a transition from one state to another, driven by interactions between these accounts.

This account-based architecture was a direct solution to the limitations of value-blindness and lack of state inherent in Bitcoin's UTXO model, enabling the stateful, multi-stage contracts that were previously impossible.

3.1 The Two Types of Accounts

Ethereum features two distinct types of accounts, each with a unique purpose and control mechanism.

Externally Owned Account (EOA) / User Account	**Contract Account / Smart Contract**
Controlled by a private key.	Controlled by its contract code.
Represents a regular user account.	Represents a smart contract.
Activated by creating and signing a transaction.	Activated when it receives a message or transaction.
Can create new contracts.	Can create other contracts via its code.

Table 3.1: Comparison of Ethereum Account Types

3.2 The Structure of an Account

Every account in Ethereum, whether an EOA or a contract, is defined by four fields (see figure 3.1):

1. **Nonce:** This field acts as a counter of transactions sent from the account. It is distinct from the nonce used in Proof-of-Work. Its purpose is to ensure that each transaction can only be processed once and that transactions are processed in the correct sequential order.

2. **Balance:** This field stores the account's current balance in Ether or its sub-denominations, such as Wei (10^{-18} ETH) or Gwei (10^{-9} ETH).

3. **Contract Code:** For a contract account, this field contains the compiled smart contract code. For an EOA, this field is empty.

4. **Storage:** This is the account's persistent storage, organized as a key-value store. It is empty by default and is only used by contract accounts to maintain state.

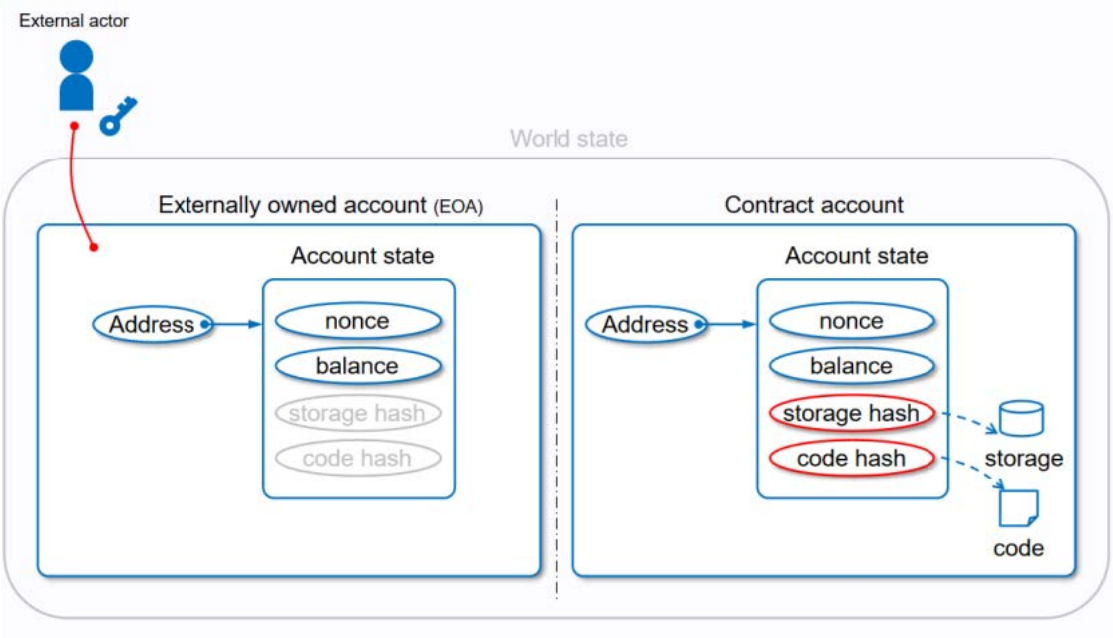

Figure 3.1: Ethereum accounts

Changes to these account fields are driven by transactions initiated by external actors and messages generated by contracts.

4 The Mechanics of Execution: Transactions, Messages, and Gas

Transactions are the fundamental drivers of all state changes in the Ethereum network. This section deconstructs the components of a transaction, explains how contracts communicate with each other, and details the critical resource management mechanism known as Gas.

4.1 Transactions: The Catalyst for State Change

A transaction in Ethereum is a signed data package that stores a message sent from an account. It contains several key fields (see figure 4.1):

- **Signature:** Identifies the sender of the transaction. In Ethereum, the signature is represented by the three values `v`, `r`, and `s`, which are the components of an ECDSA (Elliptic Curve Digital Signature Algorithm) signature:
  - `r`**:** The first component of the ECDSA signature.
  - `s`**:** The second component of the ECDSA signature, computed from the transaction hash, the sender's private key, and an ephemeral cryptographic nonce.
  - `v`**:** The recovery identifier, used to recover the sender's public key from the signature. In legacy Ethereum transactions it was typically `27` or `28`; with EIP-155 it also encodes the chain ID to protect against replay attacks across different chains.
- **to:** The 20-byte destination address.
- **value:** The amount of Ether to transfer from the sender to the recipient.
- **data:** An optional field that can contain data for contract interaction.
- **nonce:** The anti-replay value, which corresponds to the sending account's nonce.
- **gasLimit (or startgas):** The maximum number of computational steps the transaction and all its sub-executions are allowed to take.
- **gasPrice:** The fee, in Ether, that the sender agrees to pay for each computational step.

There are three primary types of transactions:

1. **Regular transactions:** A simple transfer of Ether from one account to another.
2. **Contract execution:** A transaction sent to a contract account's address, which triggers the execution of the contract's code.
3. **Contract deployment:** A transaction where the `to` field is empty (or a null address) and the `data` field contains the compiled bytecode of a smart contract. Upon execution, this transaction creates a new contract account, populates it with the code, and assigns it a unique address.

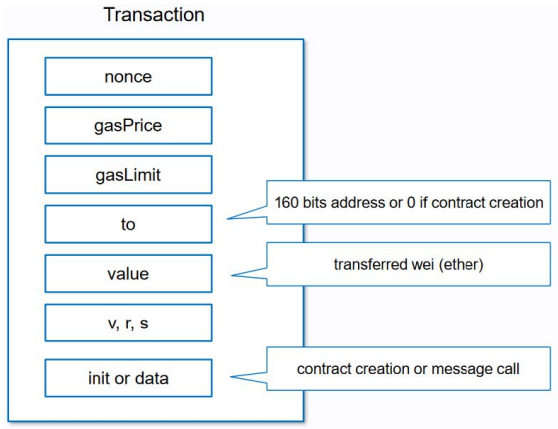

Figure 4.1: Ethereum transaction data

4.1.1 Clarifying the transaction signature

In Ethereum, a transaction is authenticated through an ECDSA signature generated by the sender. The signature consists of three values: `r`, `s`, and `v`.

Consider the following example signature associated with a transaction:

- `r = 0x1c3a...8f21`
- `s = 0x5b72...c914`
- `v = 28`

The signature is generated as follows:

1. The sender prepares the transaction fields such as the recipient address, value, data, nonce, and gas parameters.
2. The sender computes the hash of the transaction data.
3. Using their private key, the sender signs this hash with the ECDSA algorithm over the `secp256k1` curve.
4. The signing algorithm produces the values `r` and `s`.
5. An additional value, `v`, is included as a recovery identifier.

When the transaction is received by a node, the signature is interpreted as follows:

1. The node recomputes the hash of the received transaction.
2. The node uses `v`, `r`, and `s` to recover the sender's public key.
3. The Ethereum address is derived from the recovered public key.
4. The node verifies that the signature is valid and that the recovered address corresponds to the claimed sender.

Therefore, the tuple `(v, r, s)` allows Ethereum nodes to verify both the authenticity and the integrity of the transaction without requiring the sender to explicitly transmit their public key.

Operation	Gas	GasCost
PUSH1	111741	3
PUSH1	111738	3
MSTORE	111726	12
CALLDATASIZE	111724	2
ISZERO	111721	3
PUSH2	111718	3
JUMPI	111708	10

Figure 4.2: Example of EVM execution trace associating to each operation the current gas of the execution and the gas consumed by each operation

4.2 Messages: Contract-to-Contract Communication

Messages are similar to transactions (they contain an implicit sender, `to`, `value`, optionally `data` and `startgas`) but are produced by contracts, not by external actors. When one smart contract needs to interact with another, it does so by sending a message (for example, using the `CALL` opcode in the EVM). Crucially, messages are **virtual objects** that exist only within the Ethereum execution environment. They are never serialized or stored on the blockchain itself.

4.3 Gas: The Fuel for the Ethereum Network

Gas is the unit that measures the amount of computational effort required to execute operations on the Ethereum network. It serves two primary purposes:

1. **Resource Payment:** To ensure that all consumed resources—including computation, bandwidth, and storage—are paid for by the transaction sender.
2. **Preventing Wastage:** To prevent infinite loops or other forms of computational waste by imposing a hard limit (`gasLimit`) on the total computation a transaction can perform.

The `transaction fee` is calculated by multiplying the gas price by the amount of gas consumed (`gasPrice * consumedGas`). If a transaction's execution requires more computation than the specified `gasLimit`, it triggers an "Out of gas exception". In this event, the transaction is reverted, but the sender is still required to pay for the gas that was consumed up to the limit.

These mechanics of transactions, messages, and gas are formalized in the Ethereum state transition function, which governs how the blockchain's state evolves.

4.3.1 Clarification of figure 4.2

The table represents an execution trace of the Ethereum Virtual Machine (EVM). Each row corresponds to an opcode executed by the EVM, such as `PUSH1`, `MSTORE`, or `JUMPI`. The `GasCost` column indicates the amount of gas consumed by that specific instruction, while the `Gas` column indicates the amount of gas remaining at that point during execution.

Therefore, the table should not be interpreted as showing the sender's gas directly, but rather the gas budget still available to the EVM while executing the transaction or contract call. This gas budget initially comes from the transaction's gas limit and is reduced step by step as each opcode is executed.

For example, if one row shows `PUSH1` with `GasCost = 3`, and the following row reports that the remaining gas has decreased by 3, this means that the execution of that opcode consumed exactly 3 units of gas. In this sense, the trace provides a detailed view of how gas is spent throughout the execution of the bytecode.

It is also important to note that the gas cost of an opcode is not always fixed. While some instructions have a constant cost, others may have a variable cost depending on the execution context, for example due to storage access or memory expansion. Memory expansion in the EVM refers to the increase of the temporary linear memory used during contract execution. When an instruction accesses a memory region beyond the highest previously used offset, the EVM expands the memory and charges an additional gas cost. Therefore, the cost of some opcodes depends not only on the instruction itself, but also on whether executing it requires allocating more memory.

5 The Ethereum State Transition Function

At its core, every blockchain is a state transition system. The state of Ethereum is formally defined as the complete set of all existing accounts and their properties (balance, nonce, code, and storage). State changes are formalized by the Ethereum state transition function, represented as:

$$APPLY(S, TX) \rightarrow S'$$

This equation signifies that a new state `S'` is produced by applying a transaction `TX` to the current state `S` (see figure 5.1).

5.1 The State Transition Process

The state transition function follows a precise, step-by-step process for each transaction:

1. **Validation:** The function first checks whether the transaction is well-formed, for example whether the signature is valid and whether the transaction format is correct. Some preliminary checks may also be performed locally by a wallet or client before the transaction is broadcast. However, checks that depend on the current blockchain state, such as the sender's balance or nonce, can only be definitively verified by network nodes against the current state. Local preliminary checks do not consume gas on-chain.
2. **Fee Pre-payment:** The maximum transaction fee is determined from the gas limit and the fee parameters of the transaction. In legacy transactions this corresponds to `startgas * gasprice`. This maximum amount is reserved from the sender's balance, and the sender's account nonce is incremented. If the sender's balance is insufficient to cover the maximum possible cost and the transferred value, the transaction is rejected.
3. **Gas Initialization:** A `gas` counter for the transaction is initialized to the `startgas` (or `gasLimit`) value. A preliminary intrinsic gas cost, based on the transaction itself (for example its base cost and the size of its input data), is immediately deducted.
4. **Execution:** The transaction is executed by the EVM. If the transaction transfers Ether to an externally owned account, the value is transferred directly. If the recipient is a contract account, the contract code is executed until it either completes successfully, triggers a revert, or runs out of gas. During execution, each opcode consumes gas according to its computational cost.
5. **Finalization (Success):** If execution completes successfully, the unused gas is refunded to the sender. The sender ultimately pays only for the gas that was actually consumed. The corresponding transaction fee is then distributed according to the protocol rules.
6. **Finalization (Failure):** If execution fails, for example because of an exception, a revert, or an out-of-gas condition, the state changes produced by execution are reverted. However, the transaction itself remains recorded on the blockchain if it has been included in a block, the sender's nonce remains incremented, and the gas consumed during execution is still

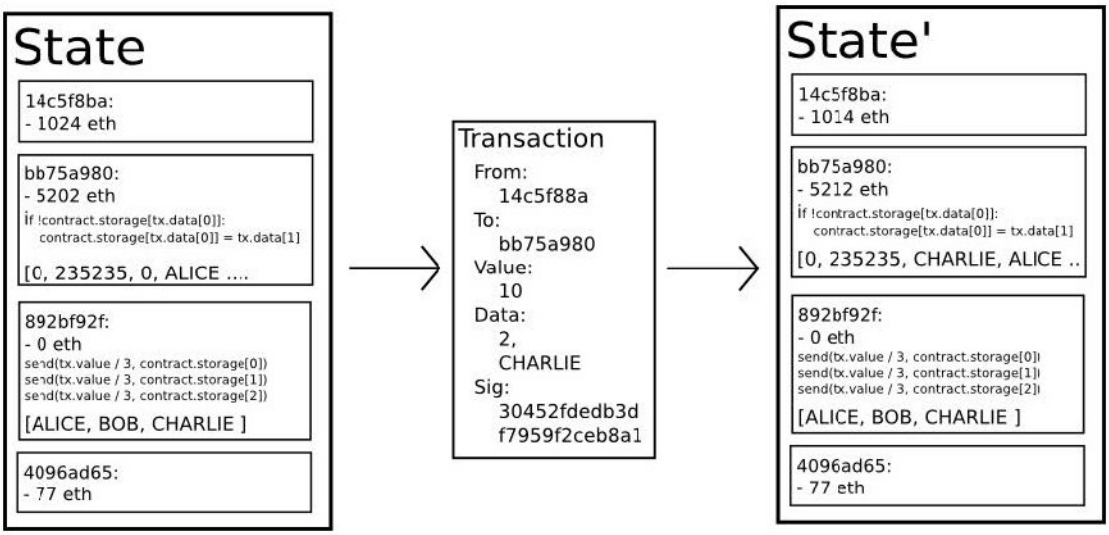

Figure 5.1: Ethereum state transitions

charged. Thus, a failed transaction may have no intended effect on the application state, while still costing Ether in transaction fees.

It is important to distinguish between *gas* and *Ether*. Gas is not a currency stored in the network, but a unit used to measure computational effort in the EVM. Transaction fees are paid in Ether according to the amount of gas consumed. Therefore, gas itself is not transferred between accounts and there is no fixed "quantity of gas" in the system.

More precisely, the transaction fee is computed from the gas used and the fee parameters of the transaction. In modern Ethereum, part of the fee may be burned by the protocol, while another part may be assigned to the block proposer as compensation. Consequently, the total amount of Ether in circulation is not constant merely because transaction fees are paid within the network.

6 Data Structures: Blocks and Merkle-Patricia Trees

The integrity and efficiency of the Ethereum blockchain depend on specific data structures designed for cryptographic verification. This section examines the composition of an Ethereum block and the crucial role of the Merkle-Patricia tree in securing state and transaction data.

6.1 The Structure of an Ethereum Block

An Ethereum block consists of a header and a list of transactions (see figure 6.1). The header contains several key fields:

- **Parent's Block Hash:** A hash that links the current block to the previous block, forming the chain.
- **Uncle Block's Hash:** The hash of an "uncle block." An uncle block is a block that was mined at the same time as the parent but was not included in the chain. Including its hash serves protocol-related purposes such has not wasting computational work to make the system fairer. The miner of the uncle block is partially rewarded for their computational effort and also the miner of the main block is rewarded for including the hash of the uncle block in the header.
- **State Root Hash:** The root hash of the Merkle-Patricia tree that represents the entire state of the system *after* all transactions in the block have been executed.
- **Transaction Root Hash:** The root hash of the Merkle-Patricia tree containing all transactions included in the block.

The list of transactions itself is also included in the block, encoded using Recursive Length Prefix Encoding (RLP).

6.2 The Merkle-Patricia Tree

The **Merkle-Patricia tree** (also known as a radix tree or trie) is the core data structure used in Ethereum to store and verify key-value mappings for both the state and transactions. Its key features are:

- It is a type of prefix tree where the path to a value is determined by its key (e.g., an account address determines the path to that account's data).
- Each node in the tree is referenced by its cryptographic hash.
- The single **root node's hash** serves as a unique cryptographic proof, or fingerprint, of the entire dataset contained within the tree.

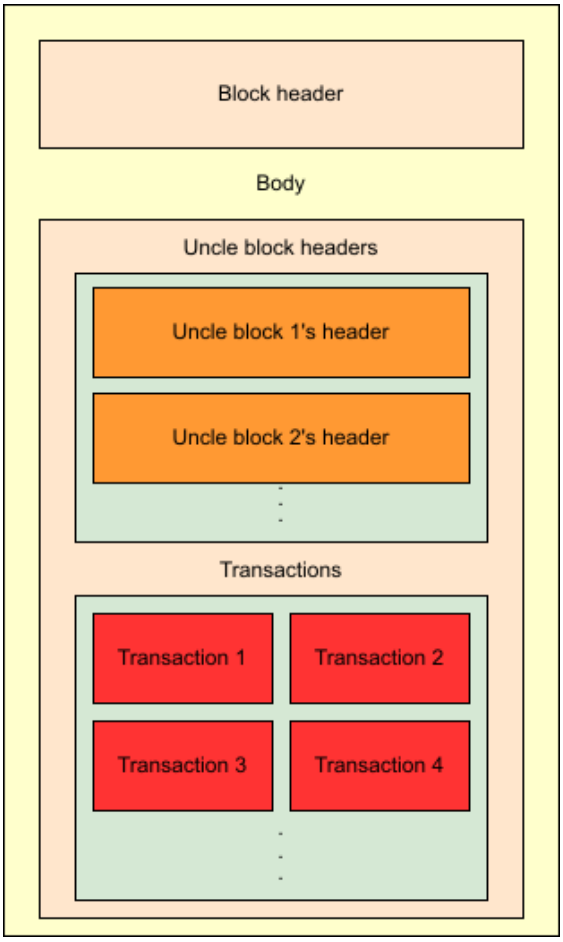

Figure 6.1: Ethereum block

The significance of this structure is profound. Any change to a single piece of data, such as an account's balance, will propagate up the tree and result in a different root hash. This makes it extremely efficient for nodes to verify that the state has changed between blocks by simply comparing a single 32-byte root hash.

7 Defining Token Systems on Ethereum

Formally, a token system is a database, backed by the blockchain, defined by a single fundamental operation: subtracting a number of units from account A and adding the same number to account B. This operation is governed by two essential conditions:

1. The sending account (A) must possess at least the number of units being transferred.
2. The transaction must be explicitly approved by the sender (A).

While this sounds simple, it enables a wide array of applications. The logic for these systems is encoded within smart contracts, allowing for the creation of diverse digital assets and functionalities.

- **Sub-currencies:** Tokens can represent other assets, such as fiat currency (e.g., Euros), commodities (e.g., gold), or financial instruments like company stocks.
- **Smart Property:** Individual, unique tokens can represent ownership of real-world property, creating a secure and verifiable digital title.
- **Unforgeable Coupons:** Secure, verifiable digital coupons that can be issued and redeemed on the blockchain.
- **Incentivization Systems:** Points systems can be created for gamification, rewards programs, or other incentivization models.

7.1 Token Standards: A Framework for Interoperability

To ensure that the vast number of tokens created on Ethereum can interact seamlessly, the community has developed token standards. These standards are strategic frameworks that allow developers to build interoperable applications—such as wallets and decentralized exchanges—that can regulate the exchange of different tokens in a uniform and predictable way. The two most prominent standards are ERC-20 for fungible tokens and ERC-721 for non-fungible tokens.

7.1.1 ERC-20: The Standard for Fungible Tokens

ERC-20 is the most widespread token standard on Ethereum, defining the interface for **fungible tokens**. Fungibility means that each unit of a token is identical and interchangeable with any other unit, much like how one banknote of a certain value is interchangeable with any other banknote of the same value.

The name "ERC-20" originates from its status as the 20th "Ethereum Request for Comments," a community process for proposing protocol improvements. This system has since evolved into the "Ethereum Improvement Proposal" (EIP) framework, but the original name for this popular standard has remained.

The ERC-20 interface specifies a set of core functions that a compliant smart contract must implement. These functions are listed in table 7.1.

Function	Description
`name()`	Returns the name of the token.
`symbol()`	Returns the symbol of the token.
`decimals()`	Returns the number of decimals the token uses.
`totalSupply()`	Returns the total supply of the token.
`balanceOf(address _owner)`	Returns the account balance of a specified owner.
`transfer(address _to, uint256 _value)`	Transfers a specified amount of tokens to a destination address.

Table 7.1: ERC-20 Interface Functions

7.1.2 ERC-721: The Standard for Non-Fungible Tokens (NFTs)

ERC-721 is the official interface for **non-fungible tokens (NFTs)**. Unlike fungible tokens, each NFT is unique and not interchangeable. This standard gained prominence with the CryptoKitties game and is now widely used to represent ownership of unique digital or physical assets, such as digital art, collectibles, or in-game items.

The ERC-721 standard includes functions for tracking ownership and managing transfers of these unique assets. These functions are listed in table 7.2.

7.2 The Evolution of the Ethereum Protocol

Ethereum is not a static protocol. It undergoes significant updates to address critical challenges, most notably scalability and energy consumption, ensuring its long-term viability and performance.

7.2.1 The Merge: Transition to Proof of Stake

"The Merge" was a major protocol update where Ethereum transitioned its consensus mechanism from Proof of Work (PoW) to Proof of Stake (PoS). This monumental shift was driven by two primary goals: increasing transaction throughput and drastically reducing the computational costs and energy consumption associated with network security.

This update was successfully completed on **September 15, 2022**. All blocks from number 15537393 onwards have been created using the PoS consensus model.

7.2.2 Sharding: A Proposed Solution for Scalability

Sharding is a proposed solution designed to address Ethereum's scalability issues. The core mechanism involves dividing the entire network into smaller, more manageable subnetworks called "shards." In this model, each node is only required to store and manage a fraction of the entire blockchain (its assigned shard) rather than the complete dataset.

This approach presents a trade-off between scalability and complexity, with several key benefits and drawbacks.

Function	Description
`balanceOf(address _owner)`	Returns the account balance of a specified owner.
`ownerOf(uint256 _tokenId)`	Returns the address of the owner of a specific NFT.
`transferFrom(address _from, address _to, uint256 _tokenId)`	Transfers ownership, but the caller is responsible for confirming the recipient's reception capability. If the recipient is not capable of receiving the NFT, it may be permanently lost.
`safeTransferFrom(address _from, address _to, uint256 _tokenId [, bytes data])`	Transfers ownership of an NFT and performs checks to ensure the recipient can receive it.
`approve(address _approved, uint256 _tokenId)`	Grants another address permission to manage a specific NFT.
`getApproved(uint256 _tokenId)`	Retrieves the approved address for a specific NFT.
`setApprovalForAll(address _operator, bool _approved)`	Enables or disables a third-party "operator" to manage all of the caller's assets.
`isApprovalForAll(address _owner, address _operator)`	Queries if an address is an authorized operator for another address.

Table 7.2: ERC-721 Interface Functions

- **Benefit:** Allows the network to scale up as it grows by significantly reducing the storage and processing requirements for individual nodes.
- **Drawback:** Requires additional coordination algorithms to manage communication and state consistency across shards, which can introduce latency.
- **Drawback:** The increased complexity and potential latency may negatively affect the usability and performance of certain applications.
- **Drawback:** The sharded architecture may not be compatible with all existing applications, potentially requiring them to be redesigned.